

MongoDB Interview Guide



ChatGPT said:

MongoDB Code Output Type Interview Questions

These questions are commonly asked in interviews where the interviewer gives code and asks:

- What will be the output?
- Which documents will be returned?
- Will this query work?
- What changes happen in DB?

1. insertOne() Output

Question

```
JavaScriptdb.users.insertOne({  
  name: "Aamir",  
  age: 22  
});
```

Output

```
JavaScript{
  acknowledged: true,
  insertedId: ObjectId("...")
}
```

Explanation

- One document inserted
- MongoDB automatically creates `_id`

2. find() Output

Collection Data

```
JSON[
  {name: "Aamir", age: 22},
  {name: "Ali", age: 25}
]
```

Question

```
JavaScriptdb.users.find();
```

Output

```
JSON[
  {name: "Aamir", age: 22},
  {name: "Ali", age: 25}
]
```

3. findOne() Output

Question

```
JavaScriptdb.users.findOne({age: 25});
```

Output

```
JSON{name: "Ali", age: 25}
```

Important

Returns only first matching document.

4. Projection Output

Question

```
JavaScriptdb.users.find({}, {name: 1, _id: 0});
```

Output

```
JSON[  
  {name: "Aamir"},  
  {name: "Ali"}  
]
```

5. \$gt Operator Output

Question

```
JavaScriptdb.users.find({age: {$gt: 22}});
```

Output

```
JSON[  
  {name: "Ali", age: 25}  
]
```

6. updateOne() Output

Before

```
JSON{name: "Aamir", age: 22}
```

Question

```
JavaScriptdb.users.updateOne(  
  {name: "Aamir"},  
  {$set: {age: 23}}  
);
```

Output

```
JavaScript{  
  acknowledged: true,  
  matchedCount: 1,  
  modifiedCount: 1  
}
```

After Update

```
JSON{name: "Aamir", age: 23}
```

7. \$inc Output

Before

```
JSON{name: "Ali", salary: 5000}
```

Question

```
JavaScriptdb.users.updateOne(  
  {name: "Ali"},  
  {$inc: {salary: 1000}}  
);
```

Output

```
JSON{name: "Ali", salary: 6000}
```

8. deleteOne() Output

Question

```
JavaScriptdb.users.deleteOne({name: "Ali"});
```

Output

```
JavaScript{  
  acknowledged: true,  
  deletedCount: 1  
}
```

9. sort() Output

Data

```
JSON[  
  {name: "A", age: 30},  
  {name: "B", age: 20},  
  {name: "C", age: 25}  
]
```

Question

```
JavaScriptdb.users.find().sort({age: 1});
```

Output

```
JSON[  
  {name: "B", age: 20},  
  {name: "C", age: 25},  
  {name: "A", age: 30}  
]
```

10. limit() Output

Question

```
JavaScriptdb.users.find().limit(2);
```

Output

Returns first 2 documents.

11. skip() + limit() Output

Data

```
JSON[
  {name: "A"},
  {name: "B"},
  {name: "C"},
  {name: "D"}
]
```

Question

```
JavaScriptdb.users.find().skip(1).limit(2);
```

Output

```
JSON[
  {name: "B"},
  {name: "C"}
]
```

12. \$or Operator Output

Question

```
JavaScriptdb.users.find({
  $or: [
    {age: 20},
    {name: "Aamir"}
  ]
});
```

Output

Returns:

- Users with age 20 OR
- Users named Aamir

13. \$in Operator Output

Question

```
JavaScriptdb.users.find({
  age: {$in: [20, 25]}
});
```

Output

Returns users whose age is 20 or 25.

14. Aggregation Output

Data

```
JSON[
  {customer: "A", amount: 100},
  {customer: "A", amount: 200},
  {customer: "B", amount: 300}
]
```

Question

```
JavaScriptdb.orders.aggregate([
  {
    $group: {
      _id: "$customer",
      total: {$sum: "$amount"}
    }
  }
]);
```

Output

```
JSON[
  { _id: "A", total: 300},
  { _id: "B", total: 300}
]
```

15. createIndex() Output

Question

```
JavaScriptdb.users.createIndex({email: 1});
```

Output

```
JavaScriptemail_1
```

16. distinct() Output

Data

```
JSON[
  {city: "Mumbai"},
  {city: "Delhi"},
  {city: "Mumbai"}
]
```

Question

```
JavaScriptdb.users.distinct("city");
```

Output

```
JavaScript["Mumbai", "Delhi"]
```

17. countDocuments() Output

Question

```
JavaScriptdb.users.countDocuments({age: {$gt: 20}});
```

Output

Returns count of users whose age > 20.

Example:

```
JavaScript2
```

18. find + AND Condition Output

Question

```
JavaScriptdb.users.find({  
  age: {$gt: 20},  
  city: "Mumbai"  
});
```

Output

Returns users:

- age > 20 AND

- city = Mumbai

19. replaceOne() Output

Before

```
JSON{name: "Aamir", age: 22}
```

Question

```
JavaScriptdb.users.replaceOne(  
  {name: "Aamir"},  
  {name: "Ali"}  
);
```

Output

```
JSON{name: "Ali"}
```

Important

Entire document replaced except `_id`.

20. upsert Output

Question

```
JavaScriptdb.users.updateOne(  
  {name: "Zaid"},  
  {$set: {age: 21}},  
  {upsert: true}  
);
```

Output

If user doesn't exist:

- New document inserted

Inserted document:

```
JSON{name: "Zaid", age: 21}
```

21. Array Push Output

Before

```
JSON{name: "Aamir", skills: ["JS"]}
```

Question

```
JavaScriptdb.users.updateOne(  
  {name: "Aamir"},  
  {$push: {skills: "MongoDB"}}  
);
```

Output

```
JSON{  
  name: "Aamir",  
  skills: ["JS", "MongoDB"]  
}
```

22. Array Pull Output

Before

```
JSON{  
  name: "Aamir",  
  skills: ["JS", "MongoDB"]  
}
```

Question

```
JavaScriptdb.users.updateOne(  
  {name: "Aamir"},  
  {$pull: {skills: "JS"}}  
);
```

Output

```
JSON{
  name: "Aamir",
  skills: ["MongoDB"]
}
```

23. Mongoose save() Output

Code

```
JavaScriptconst user = new User({
  name: "Aamir"
});

await user.save();
```

Output

- Document saved in MongoDB
- `_id` automatically generated

24. populate() Output

Data

User

```
JSON{
  name: "Aamir",
  post: ObjectId("123")
}
```

Post

```
JSON{
  _id: "123",
  title: "MongoDB"
}
```

Question

```
JavaScriptUser.find().populate("post");
```

Output

```
JSON{
  name: "Aamir",
  post: {
    _id: "123",
    title: "MongoDB"
  }
}
```

25. Interview Trick Question

Question

```
JavaScriptdb.users.find({age: "22"});
```

But database stores:

```
JSON{age: 22}
```

Output

```
JavaScript[]
```

Why?

MongoDB is type-sensitive:

- "22" → String

- 22 → Number

26. Null vs Missing Field

Data

```
JSON{name: "A"}  
{name: "B", age: null}
```

Question

```
JavaScriptdb.users.find({age: null});
```

Output

Returns BOTH:

```
JSON{name: "A"}  
{name: "B", age: null}
```

27. \$exists Output

Question

```
JavaScriptdb.users.find({  
  age: {$exists: true}  
});
```

Output

Returns documents where age field exists.

28. find().pretty() Output

Question

```
JavaScriptdb.users.find().pretty();
```

Output

Same documents but formatted nicely.

29. Explain Output

Question

```
JavaScriptdb.users.find({age: 22}).explain();
```

Output

Returns:

- Query plan
- Index used
- Execution stats

30. Most Important Interview Trick

Question

```
JavaScriptdb.users.updateMany(  
  {},  
  {$set: {age: 25}}  
);
```

Output

ALL documents age becomes 25.

Frequently Asked Output-Based Topics

- CRUD outputs
- Aggregation outputs
- Projection outputs
- Array updates
- Sorting & pagination

- Mongoose populate
- updateOne vs replaceOne
- Null handling
- Index behavior
- Type matching

Best Practice for Interviews

When interviewer gives MongoDB code:

Step 1

Check:

- Collection data
- Query condition

Step 2

Check:

- Operator
- Projection
- Update type

Step 3

Think:

- Single document or multiple?
- Type matching?
- Array or object?

Official Practice Resources

- [MongoDB Playground](#)
- [MongoDB Docs CRUD Guide](#)

You said:

CRUD outputs Aggregation outputs Projection outputs Array updates Sorting & pagination Mongoose populate updateOne vs replaceOne Null handling Index behavior Type matching

ChatGPT said:

MongoDB Important Output-Based Interview Questions

1. CRUD Outputs

CREATE Output

Question

```
JavaScriptdb.users.insertOne({
  name: "Aamir",
  age: 22
});
```

Output

```
JavaScript{
  acknowledged: true,
  insertedId: ObjectId("...")
}
```

READ Output

Data

```
JSON[
  {name: "Aamir", age: 22},
  {name: "Ali", age: 25}
]
```

Question

```
JavaScriptdb.users.find({age: 22});
```

Output

```
JSON[
  {name: "Aamir", age: 22}
]
```

UPDATE Output

Before

```
JSON{name: "Aamir", age: 22}
```

Question

```
JavaScriptdb.users.updateOne(
  {name: "Aamir"},
  {$set: {age: 23}}
);
```

Output

```
JavaScript{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

After Update

```
JSON{name: "Aamir", age: 23}
```

DELETE Output

Question

```
JavaScriptdb.users.deleteOne({name: "Ali"});
```

Output

```
JavaScript{  
  acknowledged: true,  
  deletedCount: 1  
}
```

2. Aggregation Outputs

Data

```
JSON[  
  {customer: "A", amount: 100},  
  {customer: "A", amount: 200},  
  {customer: "B", amount: 300}  
]
```

Question

```
JavaScriptdb.orders.aggregate([  
  {  
    $group: {  
      _id: "$customer",  
      total: {$sum: "$amount"}  
    }  
  }  
]);
```

Output

```
JSON[  
  { _id: "A", total: 300},  
  { _id: "B", total: 300}  
]
```

3. Projection Outputs

Data

```
JSON{
  name: "Aamir",
  age: 22,
  city: "Mumbai"
}
```

Question

```
JavaScriptdb.users.find({}, {name: 1, _id: 0});
```

Output

```
JSON[
  {name: "Aamir"}
]
```

Question 2

```
JavaScriptdb.users.find({}, {age: 0});
```

Output

```
JSON[
  {
    _id: ObjectId("..."),
    name: "Aamir",
    city: "Mumbai"
  }
]
```

4. Array Updates Outputs

\$push

Before

```
JSON{
  name: "Aamir",
  skills: ["JS"]
}
```

Question

```
JavaScriptdb.users.updateOne(
  {name: "Aamir"},
  {$push: {skills: "MongoDB"}}
);
```

Output

```
JSON{
  name: "Aamir",
  skills: ["JS", "MongoDB"]
}
```

\$pull

Before

```
JSON{
  skills: ["JS", "MongoDB"]
}
```

Question

```
JavaScriptdb.users.updateOne(
  {},
  {$pull: {skills: "JS"}}
);
```

Output

```
JSON{
  skills: ["MongoDB"]
}
```

\$addToSet

Before

```
JSON{
  skills: ["JS"]
}
```

Question

```
JavaScriptdb.users.updateOne(
  {},
  {$addToSet: {skills: "JS"}}
);
```

Output

```
JSON{
  skills: ["JS"]
}
```

Why?

Duplicate value not added.

5. Sorting & Pagination Outputs

Sorting

Data

```
JSON[
  {name: "A", age: 30},
  {name: "B", age: 20},
  {name: "C", age: 25}
]
```

Question

```
JavaScriptdb.users.find().sort({age: 1});
```

Output

```
JSON[
  {name: "B", age: 20},
  {name: "C", age: 25},
  {name: "A", age: 30}
]
```

Descending Sort

```
JavaScriptdb.users.find().sort({age: -1});
```

Output

```
JSON[
  {name: "A", age: 30},
  {name: "C", age: 25},
  {name: "B", age: 20}
]
```

Pagination

Question

```
JavaScriptdb.users.find().skip(1).limit(2);
```

Output

Returns:

- Skip first document
- Return next 2 documents

6. Mongoose Populate Outputs

Mongoose

User Collection

```
JSON{
  name: "Aamir",
  post: ObjectId("123")
}
```

Post Collection

```
JSON{
  _id: "123",
  title: "MongoDB Guide"
}
```

Question

```
JavaScriptUser.find().populate("post");
```

Output

```
JSON{
  name: "Aamir",
  post: {
    _id: "123",
    title: "MongoDB Guide"
  }
}
```

7. updateOne vs replaceOne Outputs

updateOne()

Before

```
JSON{
  name: "Aamir",
  age: 22,
  city: "Mumbai"
}
```

Question

```
JavaScriptdb.users.updateOne(
  {name: "Aamir"},
  {$set: {age: 25}}
);
```

Output

```
JSON{
  name: "Aamir",
  age: 25,
  city: "Mumbai"
}
```

Important

Only specified field updated.

replaceOne()

Question

```
JavaScriptdb.users.replaceOne(  
  {name: "Aamir"},  
  {name: "Ali"}  
);
```

Output

```
JSON{  
  name: "Ali"  
}
```

Important

Entire document replaced.

8. Null Handling Outputs

Data

```
JSON{name: "A"}  
{name: "B", age: null}
```

Question

```
JavaScriptdb.users.find({age: null});
```

Output

```
JSON[  
  {name: "A"},  
  {name: "B", age: null}  
]
```

Why?

MongoDB treats:

- missing field
- null field

both as matching.

Only Null Values

Question

```
JavaScriptdb.users.find({  
  age: {  
    $eq: null,  
    $exists: true  
  }  
});
```

Output

```
JSON[  
  {name: "B", age: null}  
]
```

9. Index Behavior Outputs

Create Index

Question

```
JavaScriptdb.users.createIndex({email: 1});
```

Output

```
JavaScriptemail_1
```

Explain with Index

Question

```
JavaScriptdb.users.find({email: "a@gmail.com"})  
.explain("executionStats");
```

Output Includes

```
JavaScript{  
  stage: "IXSCAN"  
}
```

Meaning

MongoDB used index scan.

Without Index

Output Includes

```
JavaScript{  
  stage: "COLLSCAN"  
}
```

Meaning

Full collection scan.

10. Type Matching Outputs

Data

```
JSON{  
  age: 22  
}
```

Question 1

```
JavaScriptdb.users.find({age: 22});
```

Output

```
JSON[
  {age: 22}
]
```

Question 2

```
JavaScriptdb.users.find({age: "22"});
```

Output

```
JavaScript[]
```

Why?

MongoDB is type-sensitive.

Value	Type
22	Number
"22"	String

Bonus Trick Questions

Question

```
JavaScriptdb.users.find({
  skills: "JS"
});
```

Data

```
JSON{
  skills: ["HTML", "JS"]
}
```

Output

Document WILL match.

Why?

MongoDB checks inside arrays automatically.

Question

```
JavaScriptdb.users.find({
  skills: ["HTML", "JS"]
});
```

Output

Matches ONLY exact same array order.

Most Important Interview Concepts

Topic	Interview Importance
CRUD Output	Very High
Aggregation	Very High
Projection	High
Array Operators	High
Pagination	High
Populate	Very High
Null Handling	Trick Question
Type Matching	Trick Question

Topic	Interview Importance
Index Explain	Advanced

Best Practice for Output Questions

Always Check

1. Existing data
2. Query condition
3. Data type
4. Operator behavior
5. Single vs multiple docs
6. Array behavior
7. Projection rules

Practice Resources

- [MongoDB Playground](#)
- [MongoDB Aggregation Docs](#)
- [Mongoose Populate Docs](#)

You said:

most asked and conceptual questions and code like what is output

ChatGPT said:

Most Asked MongoDB Conceptual + Output-Based Interview Questions

These are the questions most frequently asked in:

- Freshers interviews
- Node.js + MongoDB interviews

- MERN stack interviews
- Backend developer interviews

1. What is MongoDB?

Answer

MongoDB is a NoSQL document-oriented database that stores data in BSON format.

Key Features

- Schema-less
- Fast
- Scalable
- JSON-like documents
- Supports indexing
- Aggregation framework

2. SQL vs MongoDB

SQL	MongoDB
Tables	Collections
Rows	Documents
Fixed Schema	Dynamic Schema
JOIN	\$lookup
Vertical Scaling	Horizontal Scaling

3. What is BSON?

Answer

BSON means Binary JSON.

Supports:

- String
- Number
- Array
- Boolean
- Date
- ObjectId

4. What is ObjectId?

Answer

MongoDB automatically creates unique `_id`.

Example:

```
JSON{
  _id: ObjectId("507f1f77bcf86cd799439011")
}
```

Contains

- Timestamp
- Machine ID
- Process ID
- Counter

5. What is Collection?

Answer

A collection is a group of documents.

Example:

```
JavaScriptusers  
products  
orders
```

6. What is Document?

Answer

Single record in MongoDB.

Example:

```
JSON{  
  name: "Aamir",  
  age: 22  
}
```

7. Difference Between find() and findOne()

find()

findOne()

Returns cursor Returns single document

Multiple docs First matching doc

Output Question

Data

```
JSON[  
  {name: "A"},  
  {name: "B"}  
]
```

Question

```
JavaScriptdb.users.findOne();
```

Output

```
JSON{name: "A"}
```

8. Difference Between updateOne() and replaceOne()

updateOne()

Before

```
JSON{  
  name: "Aamir",  
  age: 22,  
  city: "Mumbai"  
}
```

Question

```
JavaScriptdb.users.updateOne(  
  {name: "Aamir"},  
  {$set: {age: 25}}  
);
```

Output

```
JSON{  
  name: "Aamir",  
  age: 25,  
  city: "Mumbai"  
}
```

replaceOne()

Question

```
JavaScriptdb.users.replaceOne(  
  {name: "Aamir"},  
  {name: "Ali"}  
);
```

Output

```
JSON{  
  name: "Ali"  
}
```

Concept

Entire document replaced except `_id`.

9. Type Matching Trick Question

Data

```
JSON{  
  age: 22  
}
```

Question

```
JavaScriptdb.users.find({age: "22"});
```

Output

```
JavaScript[]
```

Why?

MongoDB is type-sensitive.

Value	Type
22	Number
"22"	String

10. Null Handling Trick Question

Data

```
JSON{name: "A"}  
{name: "B", age: null}
```

Question

```
JavaScriptdb.users.find({age: null});
```

Output

```
JSON[  
  {name: "A"},  
  {name: "B", age: null}  
]
```

Concept

MongoDB matches:

- null values
- missing fields

11. What is Projection?

Answer

Projection selects specific fields.

Question

```
JavaScriptdb.users.find({}, {name: 1, _id: 0});
```

Output

```
JSON[
  {name: "Aamir"}
]
```

12. Sorting Output Question

Data

```
JSON[
  {name: "A", age: 30},
  {name: "B", age: 20}
]
```

Question

```
JavaScriptdb.users.find().sort({age: 1});
```

Output

```
JSON[
  {name: "B", age: 20},
  {name: "A", age: 30}
]
```

13. Pagination Output Question

Question

```
JavaScriptdb.users.find().skip(1).limit(2);
```

Output

- Skip first document
- Return next 2 documents

14. Array Matching Trick Question

Data

```
JSON{
  skills: ["HTML", "JS"]
}
```

Question

```
JavaScriptdb.users.find({
  skills: "JS"
});
```

Output

```
JSON[
  {
    skills: ["HTML", "JS"]
  }
]
```

Concept

MongoDB automatically checks array elements.

15. Exact Array Match

Question

```
JavaScriptdb.users.find({
  skills: ["HTML", "JS"]
});
```

Output

Matches ONLY exact same order.

16. \$push Output Question

Before

```
JSON{
  skills: ["JS"]
}
```

Question

```
JavaScriptdb.users.updateOne(
  {},
  {$push: {skills: "MongoDB"}}
);
```

Output

```
JSON{
  skills: ["JS", "MongoDB"]
}
```

17. \$addToSet Concept

Question

```
JavaScriptdb.users.updateOne(
  {},
  {$addToSet: {skills: "JS"}}
);
```

Output

Duplicate value NOT added.

18. Aggregation Most Asked Question

Data

```
JSON[
  {customer: "A", amount: 100},
  {customer: "A", amount: 200}
]
```

Question

```
JavaScriptdb.orders.aggregate([
  {
    $group: {
      _id: "$customer",
      total: {$sum: "$amount"}
    }
  }
]);
```

Output

```
JSON[
  { _id: "A", total: 300 }
]
```

19. What is Indexing?

Answer

Indexes improve query performance.

Question

```
JavaScriptdb.users.createIndex({email: 1});
```

Output

```
JavaScriptemail_1
```

20. What is COLLSCAN vs IXSCAN?

COLLSCAN

Full collection scan.

IXSCAN

Index scan.

Question

```
JavaScriptdb.users.find({email: "a@gmail.com"})  
.explain();
```

Output May Include

```
JavaScript{  
  stage: "IXSCAN"  
}
```

Meaning

Index used.

21. populate() Most Asked Question

Mongoose

User

```
JSON{  
  name: "Aamir",  
  post: ObjectId("123")  
}
```

Post

```
JSON{  
  _id: "123",  
  title: "MongoDB"  
}
```

Question

```
JavaScriptUser.find().populate("post");
```

Output

```
JSON{
  name: "Aamir",
  post: {
    _id: "123",
    title: "MongoDB"
  }
}
```

22. What is Embedding vs Referencing?

Embedding

Store inside same document

Faster reads

Small related data

Referencing

Store ObjectId

Better normalization

Large reusable data

23. Most Asked Operator Questions

\$gt

```
JavaScriptdb.users.find({
  age: {$gt: 18}
});
```

Meaning

age > 18

\$in

```
JavaScriptdb.users.find({  
  age: {$in: [20, 25]}  
});
```

Meaning

age is 20 or 25

\$or

```
JavaScriptdb.users.find({  
  $or: [  
    {age: 20},  
    {city: "Mumbai"}  
  ]  
});
```

24. upsert Concept

Question

```
JavaScriptdb.users.updateOne(  
  {name: "Aamir"},  
  {$set: {age: 22}},  
  {upsert: true}  
);
```

Concept

- Update if exists

- Insert if not exists

25. Most Important Trick Question

Question

```
JavaScriptdb.users.updateMany(  
  {},  
  {$set: {age: 25}}  
);
```

Output

ALL documents updated.

26. Difference Between deleteOne and drop

deleteOne	drop
Deletes document	Deletes collection

27. What is Aggregation?

Answer

Aggregation processes data in stages.

Common Stages

- \$match
- \$group
- \$project
- \$sort
- \$lookup

28. What is \$lookup?

Answer

MongoDB JOIN operation.

Example

```
JavaScriptdb.orders.aggregate([
  {
    $lookup: {
      from: "users",
      localField: "userId",
      foreignField: "_id",
      as: "user"
    }
  }
]);
```

29. Explain This Output

Question

```
JavaScriptdb.users.find().limit(2);
```

Output

Returns first 2 documents only.

30. Most Asked Mongoose Questions

What is Schema?

Defines structure.

```
JavaScriptconst userSchema = new mongoose.Schema({
  name: String,
  age: Number
});
```

What is Model?

Used to interact with collection.

```
JavaScriptconst User = mongoose.model("User", userSchema);
```

Top Interview Trick Areas

Topic	Why Important
Null Handling	Very common trick
Type Matching	Common output question
Array Matching	Confusing behavior
replaceOne	Data loss concept
updateMany	Updates all docs
Projection	Syntax confusion
populate	MERN interviews
Aggregation	Advanced interviews

Best Strategy for MongoDB Interviews

Whenever interviewer gives code:

Step 1

Check existing data.

Step 2

Check:

- operator
- type
- projection
- array behavior

Step 3

Think:

- single doc?
- multiple docs?

- exact match?
- type-sensitive?

Official Resources

- [MongoDB Official Docs](#)
- [MongoDB Playground](#)
- [Mongoose Docs](#)

You said:

ChatGPT said:

MongoSH Complete Command Guide

What is MongoSH?

mongosh is the modern MongoDB shell used to interact with MongoDB databases.

Used for:

- CRUD operations
- Aggregation
- Indexing
- Database management
- Admin commands

1. Start MongoSH

Open Mongo Shell

```
Bashmongosh
```

Connect to Local Database

```
Bashmongosh "mongodb://127.0.0.1:27017"
```

Connect to MongoDB Atlas

MongoDB Atlas

```
Bashmongosh "your-atlas-url"
```

2. Database Commands

Show Databases

```
JavaScriptshow dbs
```

Create / Switch Database

```
JavaScriptuse college
```

Meaning

- Switches database
- Creates DB if not exists

Current Database

```
JavaScriptdb
```

Drop Database

```
JavaScriptdb.dropDatabase()
```

Stats of Database

```
JavaScriptdb.stats()
```

3. Collection Commands

Show Collections

```
JavaScriptshow collections
```

Create Collection

```
JavaScriptdb.createCollection("users")
```

Drop Collection

```
JavaScriptdb.users.drop()
```

Collection Stats

```
JavaScriptdb.users.stats()
```

Rename Collection

```
JavaScriptdb.users.renameCollection("students")
```

4. Insert Commands

insertOne()

```
JavaScriptdb.users.insertOne({  
  name: "Aamir",  
  age: 22  
})
```

insertMany()

```
JavaScriptdb.users.insertMany([
  {name: "Ali", age: 25},
  {name: "Zaid", age: 20}
])
```

5. Read Commands

find()

```
JavaScriptdb.users.find()
```

findOne()

```
JavaScriptdb.users.findOne()
```

Pretty Output

```
JavaScriptdb.users.find().pretty()
```

Filter Query

```
JavaScriptdb.users.find({age: 22})
```

Projection

```
JavaScriptdb.users.find({}, {name: 1, _id: 0})
```

Limit

```
JavaScriptdb.users.find().limit(2)
```

Skip

```
JavaScriptdb.users.find().skip(2)
```

Sort Ascending

```
JavaScriptdb.users.find().sort({age: 1})
```

Sort Descending

```
JavaScriptdb.users.find().sort({age: -1})
```

Count Documents

```
JavaScriptdb.users.countDocuments()
```

Distinct Values

```
JavaScriptdb.users.distinct("city")
```

6. Update Commands

updateOne()

```
JavaScriptdb.users.updateOne(  
  {name: "Aamir"},  
  {$set: {age: 23}}  
)
```

updateMany()

```
JavaScriptdb.users.updateMany(  
  {},  
  {$set: {status: "active"}}  
)
```

replaceOne()

```
JavaScriptdb.users.replaceOne(  
  {name: "Aamir"},  
  {name: "Ali"}  
)
```

Upsert

```
JavaScriptdb.users.updateOne(  
  {name: "New"},  
  {$set: {age: 20}},  
  {upsert: true}  
)
```

7. Delete Commands

deleteOne()

```
JavaScriptdb.users.deleteOne({name: "Ali"})
```

deleteMany()

```
JavaScriptdb.users.deleteMany({})
```

8. Query Operators

Comparison Operators

Operator	Meaning
\$eq	Equal
\$ne	Not Equal
\$gt	Greater Than
\$gte	Greater Than Equal

Operator	Meaning
\$lt	Less Than
\$lte	Less Than Equal
\$in	In Array
\$nin	Not In

Examples

\$gt

```
JavaScriptdb.users.find({  
  age: {$gt: 18}  
})
```

\$in

```
JavaScriptdb.users.find({  
  age: {$in: [20, 25]}  
})
```

Logical Operators

Operator	Meaning
\$and	AND
\$or	OR
\$not	NOT

Operator	Meaning
----------	---------

\$nor	NOR
-------	-----

Example

```
JavaScriptdb.users.find({
  $or: [
    {age: 22},
    {city: "Delhi"}
  ]
})
```

9. Array Commands

\$push

```
JavaScriptdb.users.updateOne(
  {name: "Aamir"},
  {$push: {skills: "MongoDB"}}
)
```

\$pull

```
JavaScriptdb.users.updateOne(
  {},
  {$pull: {skills: "JS"}}
)
```

\$addToSet

```
JavaScriptdb.users.updateOne(
  {},
  {$addToSet: {skills: "NodeJS"}}
)
```

10. Aggregation Commands

Basic Aggregate

```
JavaScriptdb.orders.aggregate([
  {
    $group: {
      _id: "$customer",
      total: {$sum: "$amount"}
    }
  }
])
```

\$match

```
JavaScriptdb.orders.aggregate([
  {
    $match: {status: "completed"}
  }
])
```

\$project

```
JavaScriptdb.users.aggregate([
  {
    $project: {
      name: 1,
      _id: 0
    }
  }
])
```

\$sort

```
JavaScriptdb.users.aggregate([
  {
    $sort: {age: -1}
  }
])
```

\$lookup (JOIN)

```
JavaScriptdb.orders.aggregate([
  {
    $lookup: {
      from: "users",
      localField: "userId",
      foreignField: "_id",
      as: "user"
    }
  }
])
```

11. Index Commands

Create Index

```
JavaScriptdb.users.createIndex({email: 1})
```

Compound Index

```
JavaScriptdb.users.createIndex({
  name: 1,
  age: -1
})
```

Text Index

```
JavaScriptdb.posts.createIndex({
  title: "text"
})
```

Get Indexes

```
JavaScriptdb.users.getIndexes()
```

Drop Index

```
JavaScriptdb.users.dropIndex("email_1")
```

12. Explain Query

```
JavaScriptdb.users.find({  
  age: 22  
}).explain("executionStats")
```

Important Output Terms

Output	Meaning
--------	---------

COLLSCAN	Full collection scan
----------	----------------------

IXSCAN	Index scan
--------	------------

13. User & Auth Commands

Create User

```
JavaScriptdb.createUser({  
  user: "admin",  
  pwd: "123456",  
  roles: ["readWrite"]  
})
```

Show Users

```
JavaScriptshow users
```

Current User

```
JavaScriptdb.getUsers()
```

14. Backup & Restore Commands

Backup

```
Bashmongodump
```

Restore

```
Bashmongorestore
```

Export JSON

```
Bashmongoexport
```

Import JSON

```
Bashmongoimport
```

15. Utility Commands

Clear Screen

```
JavaScriptcls
```

Help

```
JavaScripthelp
```

Exit MongoSH

```
JavaScriptexit
```

MongoSH Interview Commands

Most Asked Commands

Command	Purpose
show dbs	Show databases
use dbName	Switch DB
show collections	Show collections
find()	Read data
insertOne()	Insert
updateOne()	Update
deleteOne()	Delete
aggregate()	Aggregation
createIndex()	Indexing
explain()	Query performance

Important MongoSH Output Questions

Question

```
JavaScriptdb.users.find({age: "22"})
```

But DB stores:

```
JSON{age: 22}
```

Output

```
JavaScript[]
```

Why?

MongoDB is type-sensitive.

Question

```
JavaScriptdb.users.find({age: null})
```

Output

Returns:

- null fields
- missing fields

Best Practice

Always Remember

- `find()` → multiple docs
- `findOne()` → single doc
- `$set` → update fields
- `replaceOne()` → replace entire doc
- `$push` → add array item
- `$addToSet` → avoid duplicates
- `.sort()` → sorting
- `.skip()` + `.limit()` → pagination

Official Resources

- [MongoDB Shell Docs](#)
- [MongoDB CRUD Docs](#)
- [MongoDB Aggregation Docs](#)

You said:

mongodb interview code to find output

ChatGPT said:

MongoDB Interview Code → Find the Output Questions

These are real interview-style questions where interviewer asks:

“What will be the output?”

Question 1 — Basic find()

Data

```
JSON[
  {name: "Aamir", age: 22},
  {name: "Ali", age: 25}
]
```

Code

```
JavaScriptdb.users.find({age: 22})
```

Output

```
JSON[
  {name: "Aamir", age: 22}
]
```

Question 2 — findOne()

Code

```
JavaScriptdb.users.findOne()
```

Output

```
JSON{name: "Aamir", age: 22}
```

Concept

Returns first document only.

Question 3 — Projection

Data

```
JSON{
  name: "Aamir",
  age: 22,
  city: "Delhi"
}
```

Code

```
JavaScriptdb.users.find({}, {name: 1, _id: 0})
```

Output

```
JSON[
  {name: "Aamir"}
]
```

Question 4 — \$gt Operator

Data

```
JSON[
  {name: "A", age: 18},
  {name: "B", age: 25}
]
```

Code

```
JavaScriptdb.users.find({  
  age: {$gt: 20}  
})
```

Output

```
JSON[  
  {name: "B", age: 25}  
]
```

Question 5 — updateOne()

Before

```
JSON{  
  name: "Aamir",  
  age: 22  
}
```

Code

```
JavaScriptdb.users.updateOne(  
  {name: "Aamir"},  
  {$set: {age: 30}}  
)
```

Output

```
JavaScript{  
  acknowledged: true,  
  matchedCount: 1,  
  modifiedCount: 1  
}
```

After Update

```
JSON{
  name: "Aamir",
  age: 30
}
```

Question 6 — replaceOne() Trick

Before

```
JSON{
  name: "Aamir",
  age: 22,
  city: "Delhi"
}
```

Code

```
JavaScriptdb.users.replaceOne(
  {name: "Aamir"},
  {name: "Ali"}
)
```

Output

```
JSON{
  name: "Ali"
}
```

Important

Entire document replaced.

Question 7 — Type Matching Trick

Data

```
JSON{  
  age: 22  
}
```

Code

```
JavaScriptdb.users.find({age: "22"})
```

Output

```
JavaScript[]
```

Why?

- 22 → Number
- "22" → String

MongoDB is type-sensitive.

Question 8 — Null Handling Trick

Data

```
JSON{name: "A"}  
{name: "B", age: null}
```

Code

```
JavaScriptdb.users.find({age: null})
```

Output

```
JSON[
  {name: "A"},
  {name: "B", age: null}
]
```

Concept

Matches:

- null values
- missing fields

Question 9 — \$exists

Code

```
JavaScriptdb.users.find({
  age: {$exists: true}
})
```

Output

```
JSON[
  {name: "B", age: null}
]
```

Question 10 — Array Matching

Data

```
JSON{
  skills: ["HTML", "JS"]
}
```

Code

```
JavaScriptdb.users.find({
  skills: "JS"
})
```

Output

```
JSON[
  {
    skills: ["HTML", "JS"]
  }
]
```

Concept

MongoDB automatically searches inside arrays.

Question 11 — Exact Array Match

Code

```
JavaScriptdb.users.find({
  skills: ["HTML", "JS"]
})
```

Output

Matches only exact same order.

Question 12 — \$push

Before

```
JSON{
  skills: ["JS"]
}
```

Code

```
JavaScriptdb.users.updateOne(  
  {},  
  {$push: {skills: "MongoDB"}}  
)
```

Output

```
JSON{  
  skills: ["JS", "MongoDB"]  
}
```

Question 13 — \$addToSet Trick

Before

```
JSON{  
  skills: ["JS"]  
}
```

Code

```
JavaScriptdb.users.updateOne(  
  {},  
  {$addToSet: {skills: "JS"}}  
)
```

Output

```
JSON{  
  skills: ["JS"]  
}
```

Why?

Duplicate not added.

Question 14 — Sorting

Data

```
JSON[
  {name: "A", age: 30},
  {name: "B", age: 20}
]
```

Code

```
JavaScriptdb.users.find().sort({age: 1})
```

Output

```
JSON[
  {name: "B", age: 20},
  {name: "A", age: 30}
]
```

Question 15 — Pagination

Data

```
JSON[
  {name: "A"},
  {name: "B"},
  {name: "C"},
  {name: "D"}
]
```

Code

```
JavaScriptdb.users.find().skip(1).limit(2)
```

Output

```
JSON[
  {name: "B"},
  {name: "C"}
]
```

Question 16 — Aggregation

Data

```
JSON[
  {customer: "A", amount: 100},
  {customer: "A", amount: 200},
  {customer: "B", amount: 300}
]
```

Code

```
JavaScriptdb.orders.aggregate([
  {
    $group: {
      _id: "$customer",
      total: {$sum: "$amount"}
    }
  }
])
```

Output

```
JSON[
  { _id: "A", total: 300},
  { _id: "B", total: 300}
]
```

Question 17 — updateMany() Trick

Data

```
JSON[
  {name: "A"},
  {name: "B"}
]
```

Code

```
JavaScriptdb.users.updateMany(
  {},
  {$set: {age: 25}}
)
```

Output

```
JSON[
  {name: "A", age: 25},
  {name: "B", age: 25}
]
```

Question 18 — deleteOne()

Data

```
JSON[
  {name: "A"},
  {name: "B"}
]
```

Code

```
JavaScriptdb.users.deleteOne({name: "A"})
```

Output

```
JavaScript{
  acknowledged: true,
  deletedCount: 1
}
```

Question 19 — distinct()

Data

```
JSON[
  {city: "Delhi"},
  {city: "Mumbai"},
  {city: "Delhi"}
]
```

Code

```
JavaScriptdb.users.distinct("city")
```

Output

```
JavaScript["Delhi", "Mumbai"]
```

Question 20 — populate()

Mongoose

User

```
JSON{
  name: "Aamir",
  post: ObjectId("123")
}
```

Post

```
JSON{
  _id: "123",
  title: "MongoDB"
}
```

Code

```
JavaScriptUser.find().populate("post")
```

Output

```
JSON{
  name: "Aamir",
  post: {
    _id: "123",
    title: "MongoDB"
  }
}
```

Most Asked MongoDB Trick Concepts

Topic	Important Point
replaceOne	Replaces full doc
updateMany	Updates all docs
Type Matching	String $\neq$ Number
Null Handling	null + missing fields
Arrays	MongoDB auto-checks arrays
Projection	Include/exclude fields
Pagination	skip + limit
Aggregation	Grouping & calculations

Topic	Important Point
populate	Replaces ObjectId

Best Interview Strategy

Whenever interviewer gives code:

Step 1

Check existing data.

Step 2

Check:

- operator
- type
- projection
- array behavior

Step 3

Think:

- one doc or many?
- exact match?
- type-sensitive?

Practice Resources

- [MongoDB Playground](#)
- [MongoDB Docs Query Guide](#)
- [Mongoose Populate Docs](#)

You said:

mongodb interview code to find output conceptual and tough question and ans

ChatGPT said:

MongoDB Tough Interview Questions (Code + Output + Concept)

These are advanced and tricky questions asked in:

- MERN interviews
- Backend interviews
- Node.js interviews
- Experienced MongoDB interviews

1. replaceOne() Dangerous Question

Data

```
JSON{
  name: "Aamir",
  age: 22,
  city: "Delhi"
}
```

Code

```
JavaScriptdb.users.replaceOne(
  {name: "Aamir"},
  {age: 30}
)
```

Output

```
JSON{
  age: 30
}
```

Concept

replaceOne() replaces the FULL document except **_id**.

Interview Trap

People think only age updates.

2. updateOne() vs replaceOne()

Code 1

```
JavaScriptdb.users.updateOne(  
  {},  
  {$set: {age: 30}}  
)
```

Output

```
JSON{  
  name: "Aamir",  
  age: 30,  
  city: "Delhi"  
}
```

Code 2

```
JavaScriptdb.users.replaceOne(  
  {},  
  {age: 30}  
)
```

Output

```
JSON{  
  age: 30  
}
```

3. Type Matching Tough Question

Data

```
JSON[
  {age: 22},
  {age: "22"}
]
```

Code

```
JavaScriptdb.users.find({age: 22})
```

Output

```
JSON[
  {age: 22}
]
```

Code 2

```
JavaScriptdb.users.find({age: "22"})
```

Output

```
JSON[
  {age: "22"}
]
```

Concept

MongoDB is STRICTLY type-sensitive.

4. Null Handling Tough Question

Data

```
JSON{name: "A"}
{name: "B", age: null}
{name: "C", age: 22}
```

Code

```
JavaScriptdb.users.find({age: null})
```

Output

```
JSON[
  {name: "A"},
  {name: "B", age: null}
]
```

Tough Concept

MongoDB treats:

- missing fields
- null fields

both as matching.

5. Only NULL Values Question

Code

```
JavaScriptdb.users.find({
  age: {
    $eq: null,
    $exists: true
  }
})
```

Output

```
JSON[
  {name: "B", age: null}
]
```

6. Array Matching Trick

Data

```
JSON{
  skills: ["HTML", "CSS", "JS"]
}
```

Code

```
JavaScriptdb.users.find({
  skills: "JS"
})
```

Output

```
JSON[
  {
    skills: ["HTML", "CSS", "JS"]
  }
]
```

Tough Concept

MongoDB automatically checks array elements.

7. Exact Array Order Match

Data

```
JSON{
  skills: ["HTML", "CSS"]
}
```

Code

```
JavaScriptdb.users.find({
  skills: ["CSS", "HTML"]
})
```

Output

```
JavaScript[]
```

Concept

Exact order required.

8. \$push vs \$addToSet

Data

```
JSON{
  skills: ["JS"]
}
```

\$push

Code

```
JavaScriptdb.users.updateOne(
  {},
  {$push: {skills: "JS"}}
)
```

Output

```
JSON{
  skills: ["JS", "JS"]
}
```

\$addToSet

Code

```
JavaScriptdb.users.updateOne(  
  {},  
  {$addToSet: {skills: "JS"}}  
)
```

Output

```
JSON{  
  skills: ["JS"]  
}
```

Tough Concept

Operator **Duplicate Allowed**

\$push Yes

\$addToSet No

9. Projection Tough Question

Data

```
JSON{  
  name: "Aamir",  
  age: 22,  
  city: "Delhi"  
}
```

Code

```
JavaScriptdb.users.find({}, {  
  name: 1,  
  city: 0  
})
```

Output

```
JavaScriptERROR
```

Concept

Cannot mix:

- include fields
- exclude fields

except `_id`.

10. updateMany() Dangerous Question

Data

```
JSON[
  {name: "A"},
  {name: "B"}
]
```

Code

```
JavaScriptdb.users.updateMany(
  {},
  {$set: {status: "active"}}
)
```

Output

```
JSON[
  {name: "A", status: "active"},
  {name: "B", status: "active"}
]
```

Tough Concept

Empty filter `{}` means ALL documents.

11. Aggregation Tough Question

Data

```
JSON[
  {dept: "IT", salary: 1000},
  {dept: "IT", salary: 2000},
  {dept: "HR", salary: 1500}
]
```

Code

```
JavaScriptdb.emp.aggregate([
  {
    $group: {
      _id: "$dept",
      total: {$sum: "$salary"},
      avg: {$avg: "$salary"}
    }
  }
])
```

Output

```
JSON[
  {
    _id: "IT",
    total: 3000,
    avg: 1500
  },
  {
    _id: "HR",
    total: 1500,
    avg: 1500
  }
]
```

12. \$lookup Tough Question

Orders

```
JSON{
  userId: 1,
  amount: 500
}
```

Users

```
JSON{
  _id: 1,
  name: "Aamir"
}
```

Code

```
JavaScriptdb.orders.aggregate([
  {
    $lookup: {
      from: "users",
      localField: "userId",
      foreignField: "_id",
      as: "user"
    }
  }
])
```

Output

```
JSON[
  {
    userId: 1,
    amount: 500,
    user: [
      {
        _id: 1,
        name: "Aamir"
      }
    ]
  }
]
```

13. Index Interview Question

Code

```
JavaScriptdb.users.find({
  email: "a@gmail.com"
}).explain()
```

Output May Include

```
JavaScript{
  stage: "COLLSCAN"
}
```

Meaning

Full collection scan.

After Index

Code

```
JavaScriptdb.users.createIndex({email: 1})
```

Output

```
JavaScriptemail_1
```

explain() Output

```
JavaScript{  
  stage: "IXSCAN"  
}
```

Tough Concept

Output	Meaning
--------	---------

COLLSCAN	Slow full scan
----------	----------------

IXSCAN	Fast indexed scan
--------	-------------------

14. countDocuments() Trick

Data

```
JSON[  
  {age: 20},  
  {age: 25},  
  {age: 30}  
]
```

Code

```
JavaScriptdb.users.countDocuments({  
  age: {$gt: 20}  
})
```

Output

```
JavaScript2
```

15. skip() + limit() Tough Question

Data

```
JSON[
  {name: "A"},
  {name: "B"},
  {name: "C"},
  {name: "D"}
]
```

Code

```
JavaScriptdb.users.find()
.skip(2)
.limit(1)
```

Output

```
JSON[
  {name: "C"}
]
```

16. Mongoose populate() Tough Concept

Mongoose

User

```
JSON{
  name: "Aamir",
  post: ObjectId("123")
}
```

Post

```
JSON{
  _id: "123",
  title: "MongoDB"
}
```

Code

```
JavaScriptUser.find().populate("post")
```

Output

```
JSON{
  name: "Aamir",
  post: {
    _id: "123",
    title: "MongoDB"
  }
}
```

Tough Concept

populate() replaces ObjectId with actual document.

17. deleteMany() Dangerous Question

Code

```
JavaScriptdb.users.deleteMany({})
```

Output

ALL documents deleted.

18. findOneAndUpdate()

Before

```
JSON{
  name: "Aamir",
  age: 22
}
```

Code

```
JavaScriptdb.users.findOneAndUpdate(  
  {name: "Aamir"},  
  {$set: {age: 30}}  
)
```

Output

By default returns OLD document.

```
JSON{  
  name: "Aamir",  
  age: 22  
}
```

To Return Updated Doc

```
JavaScriptdb.users.findOneAndUpdate(  
  {name: "Aamir"},  
  {$set: {age: 30}},  
  {new: true}  
)
```

19. Upsert Tough Question

Code

```
JavaScriptdb.users.updateOne(  
  {name: "Zaid"},  
  {$set: {age: 20}},  
  {upsert: true}  
)
```

Output

If document not found: New document inserted.

20. Most Asked Conceptual Questions

Why MongoDB?

- Flexible schema
- Fast development
- Horizontal scaling
- JSON-like documents

Embedding vs Referencing

Embedding	Referencing
Faster reads	Better normalization
Small related data	Large reusable data

What is Aggregation?

Processes documents in stages.

What is Sharding?

Splits data across multiple servers.

What is Replication?

Copies data across servers for backup/high availability.

Most Dangerous MongoDB Interview Traps

Trap	Why Important
replaceOne	Deletes fields
updateMany({})	Updates ALL docs
deleteMany({})	Deletes ALL docs
null query	Matches missing fields
array exact match	Order matters
projection mix	Causes error

Trap	Why Important
type matching	Strict types
populate	ObjectId replacement

Best Interview Strategy

When interviewer gives MongoDB code:

Step 1

Check existing data carefully.

Step 2

Check:

- operator
- projection
- array
- data types

Step 3

Think:

- update or replace?
- single or many?
- exact match?
- null handling?

Practice Resources

- [MongoDB Official Docs](#)
- [MongoDB Playground](#)
- [Mongoose Docs](#)

You said:

Express js all concept explain

ChatGPT said:

Express.js Complete Concepts Guide

What is Express.js?

Express.js is a lightweight web framework for Node.js used to build:

- Web applications
- APIs
- Backend servers

Why Use Express.js?

Without Express

In Node.js, creating routes and handling requests manually is difficult.

With Express

Express makes:

- Routing easy
- Middleware handling easy
- API creation fast
- Server management simple

1. Install Express.js

Create Project

```
Bashmkdir myapp
cd myapp
npm init -y
```

Install Express

```
Bashnpm install express
```

2. First Express Server

app.js

```
JavaScriptconst express = require("express");

const app = express();

app.listen(3000, () => {
  console.log("Server Started");
});
```

Concept

Code	Meaning
express()	Creates app
app.listen()	Starts server
3000	Port number

3. Route Handling

Routes define URLs.

GET Route

```
JavaScriptapp.get("/", (req, res) => {
  res.send("Home Page");
});
```

About Route

```
JavaScriptapp.get("/about", (req, res) => {  
  res.send("About Page");  
});
```

Dynamic Route

```
JavaScriptapp.get("/user/:id", (req, res) => {  
  console.log(req.params);  
  
  res.send("Dynamic Route");  
});
```

URL

```
/user/101
```

Output

```
JavaScript{  
  id: "101"  
}
```

4. Request Object (req)

req contains client request data.

Important Properties

Property	Meaning
req.params	URL params
req.query	Query params
req.body	Form/API data

Property	Meaning
req.headers	Headers

req.query Example

URL

```
/search?name=aamir
```

Code

```
JavaScriptapp.get("/search", (req, res) => {  
  console.log(req.query);  
});
```

Output

```
JavaScript{  
  name: "aamir"  
}
```

5. Response Object (res)

Used to send response.

res.send()

```
JavaScriptres.send("Hello");
```

res.json()

```
JavaScriptres.json({  
  name: "Aamir"  
});
```

res.render()

Used with templates.

```
JavaScriptres.render("home.ejs");
```

res.redirect()

```
JavaScriptres.redirect("/home");
```

res.status()

```
JavaScriptres.status(404).send("Not Found");
```

6. HTTP Methods

Method	Purpose
GET	Read data
POST	Create data
PUT	Update full
PATCH	Update partial
DELETE	Delete data

POST Route

```
JavaScriptapp.post("/user", (req, res) => {  
  res.send("User Created");  
});
```

DELETE Route

```
JavaScriptapp.delete("/user/:id", (req, res) => {  
  res.send("Deleted");  
});
```

7. Middleware

What is Middleware?

Middleware runs:

- before request reaches route
- between request and response

Basic Middleware

```
JavaScriptapp.use((req, res, next) => {  
  console.log("Middleware Running");  
  
  next();  
});
```

Concept

Function	Purpose
----------	---------

next()	Pass control आगे
--------	------------------

Middleware Flow

```
Request → Middleware → Route → Response
```

8. Express Built-in Middleware

JSON Middleware

```
JavaScriptapp.use(express.json());
```

Used for JSON API data.

URL Encoded Middleware

```
JavaScriptapp.use(express.urlencoded({extended: true}));
```

Used for form data.

9. Route Parameters

Code

```
JavaScriptapp.get("/product/:id", (req, res) => {  
  res.send(req.params.id);  
});
```

URL

```
/product/500
```

Output

```
500
```

10. Query Strings

URL

```
/search?category=mobile
```

Code

```
JavaScriptapp.get("/search", (req, res) => {  
  console.log(req.query);  
});
```

Output

```
JavaScript{  
  category: "mobile"  
}
```

11. Serving Static Files

Folder Structure

```
public/  
style.css
```

Code

```
JavaScriptapp.use(express.static("public"));
```

12. Template Engine

Install EJS

EJS

```
Bashnpm install ejs
```

Setup

```
JavaScriptapp.set("view engine", "ejs");
```

Render File

```
JavaScriptres.render("home");
```

EJS Example

```
HTML<h1><%= name %></h1>
```

13. Express Router

Why Router?

Separates routes into files.

userRoutes.js

```
JavaScriptconst express = require("express");

const router = express.Router();

router.get("/", (req, res) => {
  res.send("Users");
});

module.exports = router;
```

app.js

```
JavaScriptconst userRoutes = require("./userRoutes");

app.use("/users", userRoutes);
```

14. Error Handling

Basic Error Middleware

```
JavaScriptapp.use((err, req, res, next) => {
  res.status(500).send(err.message);
});
```

404 Route

```
JavaScriptapp.use((req, res) => {
  res.status(404).send("Page Not Found");
});
```

15. Async Error Handling

Problem

Async errors not automatically caught.

Example

```
JavaScriptapp.get("/", async(req, res) => {  
  throw new Error("Error");  
});
```

Solution

Use try-catch.

```
JavaScripttry {  
  
} catch(err) {  
  
}
```

16. REST API Concepts

RESTful Routes

Route	Purpose
GET /users	Get users
GET /users/:id	Get single user
POST /users	Create
PUT /users/:id	Update

Route	Purpose
DELETE /users/:id	Delete

17. CRUD Example

```
JavaScriptapp.post("/users", (req, res) => {  
  res.send("Create");  
});  
  
app.get("/users", (req, res) => {  
  res.send("Read");  
});  
  
app.put("/users/:id", (req, res) => {  
  res.send("Update");  
});  
  
app.delete("/users/:id", (req, res) => {  
  res.send("Delete");  
});
```

18. Express + MongoDB Connection

MongoDB

```
JavaScriptconst mongoose = require("mongoose");  
  
mongoose.connect("mongodb://127.0.0.1:27017/test")  
  .then(() => {  
    console.log("DB Connected");  
  });
```

19. MVC Pattern

Meaning

Part	Work
Model	Database
View	Frontend

Part Work

Controller Logic

Folder Structure

```
models/  
views/  
controllers/  
routes/
```

20. Cookies

Install

```
Bashnpm install cookie-parser
```

Setup

```
JavaScriptconst cookieParser = require("cookie-parser");  
  
app.use(cookieParser());
```

Set Cookie

```
JavaScriptres.cookie("name", "Aamir");
```

Read Cookie

```
JavaScriptreq.cookies
```

21. Session

Install

```
Bashnpm install express-session
```

Setup

```
JavaScriptconst session = require("express-session");

app.use(session({
  secret: "mysession"
}));
```

22. Authentication

Common Packages

Package	Purpose
passport	Authentication
bcrypt	Password hashing
jsonwebtoken	JWT auth

bcrypt Example

```
JavaScriptbcrypt.hash(password, 10);
```

JWT Example

```
JavaScriptjwt.sign({id: 1}, "secret");
```

23. File Upload

Install

Multer

```
Bashnpm install multer
```

Upload Example

```
JavaScriptconst multer = require("multer");

const upload = multer({
  dest: "uploads/"
});

app.post(
  "/upload",
  upload.single("image"),
  (req, res) => {
    res.send("Uploaded");
  }
);
```

24. Environment Variables

Install

dotenv

```
Bashnpm install dotenv
```

Setup

```
JavaScriptrequire("dotenv").config();
```

Use

```
JavaScriptprocess.env.SECRET_KEY
```

25. Important Interview Questions

What is Middleware?

Function between request and response.

Difference Between app.use and app.get

app.use **app.get**

All methods GET only

What is next()?

Passes control to next middleware.

Difference Between PUT and PATCH

PUT **PATCH**

Full update Partial update

What is REST API?

Standard backend API structure.

Most Important Express Concepts

Topic	Interview Importance
Routing	Very High
Middleware	Very High
REST API	Very High
Error Handling	High
MVC	High
JWT Auth	Very High
Express Router	High

Topic	Interview Importance
CRUD	Very High

Common Interview Output Questions

Question

```
JavaScriptapp.use((req,res,next)=>{  
  console.log("A");  
  next();  
});  
  
app.get("/",(req,res)=>{  
  console.log("B");  
  res.send("Home");  
});
```

Output

```
A  
B
```

Question

```
JavaScriptapp.get("/:id",(req,res)=>{  
  res.send(req.params.id);  
});
```

URL:

```
/100
```

Output

```
100
```

Official Resources

- [Express.js Official Docs](#)
- [Node.js Official Docs](#)
- [EJS Documentation](#)